МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
ІМЕНІ ІГОРЯ СІКОРСЬКОГО»

Кафедра прикладної математики

ЗВІТ ЛАБОРАТОРНОЇ РОБОТИ №1

з дисципліни «Основи геометричного моделювання»
на тему: «Побудова дороги – клотоїда – коло – пряма (біля міста)»

Виконав:
студент III курсу групи КМ-32,
спеціальність F1
Прикладна математика
Мурсюкаєв Дамір Андрійович

Київ — 2026

ЗМІСТ

ВСТУП....3
1 ПОСТАНОВКА ЗАДАЧІ....4
1.1 Мета....4
1.2 Об’єкти на карті....4
1.3 Структура дороги....4
2 ТЕОРЕТИЧНА ЧАСТИНА....6
2.1 Геометрія міста і точка дотику....6
2.2 Вхідна клотоїда....6
2.3 Кругова вставка....7
2.4 Вихідна клотоїда....7
2.5 Осьова прив’язка фігури....8
2.6 Розміщення фігури в глобальній системі координат....8
2.7 Знакова кутова функція і перетин прямих....9
2.8 Алгоритм двох фігур....9
2.9 Побудова сегмента дороги між двома містами....10
2.10 Перевірка зіткнення траси з зоною міста....11
2.11 Обхід перешкод....11
2.12 Оптимізація порядку обходу міст....12
3 ПРОГРАМНА РЕАЛІЗАЦІЯ....13
4 ПРИКЛАДИ РОБОТИ ПРОГРАМИ....15
5 АНАЛІЗ РЕЗУЛЬТАТІВ....18
6 ВИСНОВКИ....20
7 СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ....21
ДОДАТОК А....22
А.1 Побудова клотоїдної фігури....22
А.2 Побудова сегмента дороги між двома містами....22
А.3 Пошук проміжного напряму для двох фігур....23
А.4 Перевірка перетину траси з містом або перешкодою....24
А.5 Обхід озер....24
А.6 Оптимізація порядку обходу міст....26
А.7 Побудова повної дороги....27

ВСТУП

Проєктування автомобільних доріг є важливою задачею транспортної інженерії, оскільки від правильності геометрії траси залежать безпека руху, комфорт водіїв, довжина маршруту та економічна доцільність будівництва. Під час побудови дороги необхідно забезпечити плавний перехід між прямолінійними ділянками та круговими кривими. Для цього застосовуються перехідні криві, зокрема клотоїди.

У даній роботі розроблено програму, яка моделює побудову траси між кількома містами на площині. Кожне місто задається положенням, радіусом і напрямком входу/виходу дороги. Також враховується можливість наявності перешкод, зокрема озер, які необхідно обходити.

1 ПОСТАНОВКА ЗАДАЧІ

1.1 Мета

Розробити програму що автоматично будує автомобільну трасу через n міст. Траса огинає кожне місто ззовні по дотичній, використовуючи схему «пряма → клотоїда → коло → клотоїда → пряма», причому, біля міста буде саме “пряма”. Загальна довжина траси мінімізується автоматично. Користувач може змінювати координати міст, нахил дороги, сторону обходу міста, параметри побудови дороги та додавати перешкоди – траса перебудовується при повторному запуску.

1.2 Об’єкти на карті

Міста $\mathrm{M}_1, \mathrm{M}_2, \ldots, \mathrm{M}_n$ задаються координатами (x_i, y_i), радіусом забороненої зони (зони самого міста) R_i, напрямком дороги α_i і невеликим відступом для реалістичної побудови прямої дотичної ділянки дороги. Також задаються перешкоди (реалістично – озера) $\mathrm{L}_1, \mathrm{L}_2, \ldots, \mathrm{L}_n$ як кола з певними радіусами та безпечним відступом, ідентично до міст. Ці перешкоди дорога повинна обходити по прямій дотичній до безпечного інтервалу. Траса не заходить всередину жодної зони. Дорога підходить до кожного міста по прямій, що є зовнішньою дотичною до кола зони з відступом.

1.3 Структура дороги

Необхідно побудувати дорогу, що з’єднує задані міста, дотримуючись геометричних вимог до траси. У загальному випадку траса складається з:

– прямих ділянок;

– перехідних кривих типу клотоїда;

– кругових дуг.

Додатково на площині можуть бути задані озера або інші перешкоди, які дорога не повинна перетинати. Для цього необхідно перебудовувати маршрут або локально вставляти обхідну ділянку.

Результатом роботи програми є:

– послідовність геометричних елементів траси;

– графічна схема побудованої дороги;

– довжина дороги;

– демонстрація обраного режиму побудови.

Для кожного міста будується вузол обходу – фігура :

– пряма – вхідна клотоїда – дуга кола – вихідна клотоїда – пряма – **точка дотику** – пряма – вхідна клотоїда – дуга кола – вихідна клотоїда – пряма

Графічне зображення вузлу обходу, дороги, точки дотику та границь міста представлено на рис. 1.

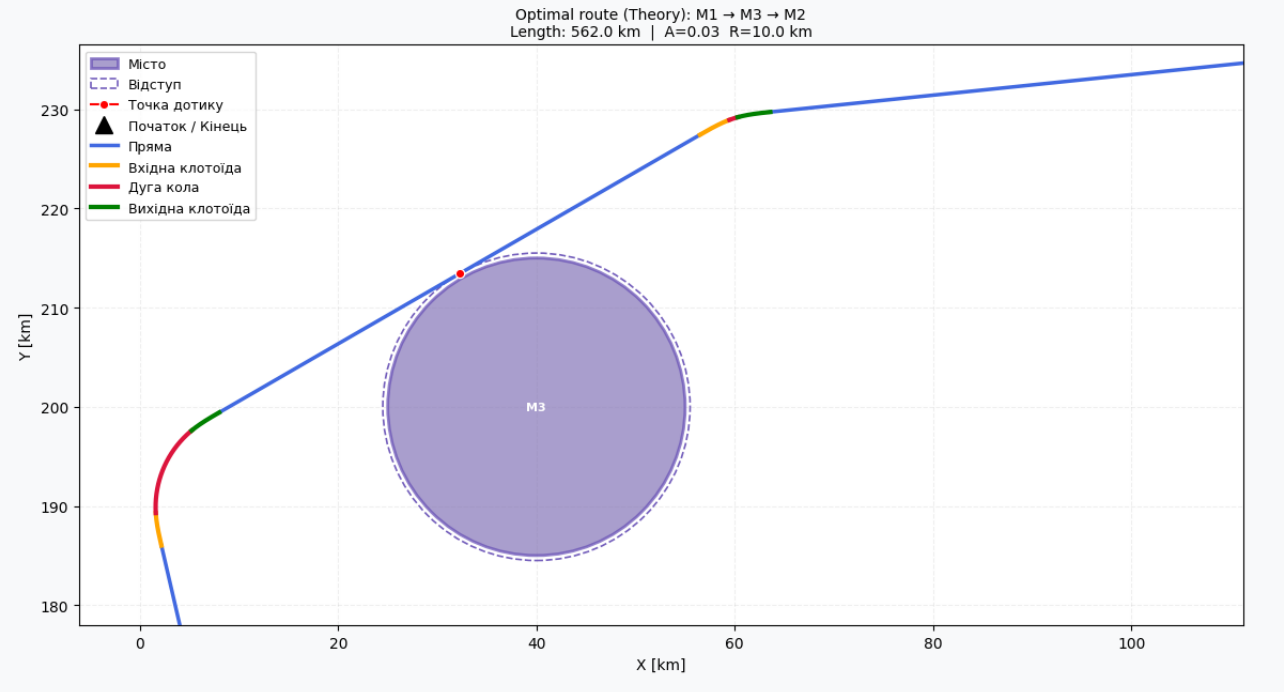


Рис. 1 – Графічне представлення дороги

2 ТЕОРЕТИЧНА ЧАСТИНА

2.1 Геометрія міста і точка дотику

Кожне місто задається набором параметрів: координати центру, радіус забороненої зони, кут осі дороги та відступ для реалістичної побудови траси. Траса не заходить всередину жодної зони і підходить до міста по прямій – зовнішній дотичній до кола зони з відступом.

Радіус контакту (відстань від центру міста до лінії траси):

$$r_i^{contact} = R_i + c,$$

де R_i – радіус зони міста, $c = 0.5$ км – відступ безпеки.

Одиничний вектор напряму дороги та ліва нормаль до нього:

$$\mathbf{t}(\alpha) = \begin{pmatrix} \cos\alpha \\ \sin\alpha \end{pmatrix}, \qquad \mathbf{n}(\alpha) = \begin{pmatrix} -\sin\alpha \\ \cos\alpha \end{pmatrix}$$

Можливі дві точки дотику:

$$T_+ = C + r^{contact}\mathbf{n}, \qquad T_- = C - r^{contact}\mathbf{n}$$

Якщо сторона обходу задана явно, використовується відповідна точка. Якщо сторона не задана, програма обирає геометрично кращий варіант.

2.2 Вхідна клотоїда

Клотоїда – крива, кривизна якої лінійно зростає від нуля до максимального значення $\frac{1}{R}$ вздовж натуральної координати $s \in [0,\ S_0]$:

$$\kappa\,(s) = A \cdot s$$

Параметр A визначає швидкість зростання кривизни:

$$A = \frac{1}{R \cdot S_0}, \qquad S_0 = \frac{1}{R \cdot A} = \frac{\lambda}{A}, \qquad \lambda = \frac{1}{R},$$

де S_0 – повна довжина клотоїдної вставки, $\lambda = \frac{1}{R}$ – максимальна кривизна.

Кут повороту дотичного вектора (фаза) в точці s:

$$\varphi(s) = \int_0^s \kappa\,(t)\; dt = \frac{As^2}{2}$$

Кут, на який повертає одна повна клотоїдна вставка:

$$\varphi_{cl} = \frac{AS_0^2}{2}$$

Координати точок клотоїди задаються інтегралами Френеля:

$$x(s) = \int_0^s \cos\left(\frac{\pi t^2}{2}\right) dt, \qquad y(s) = \int_0^s \sin\left(\frac{\pi t^2}{2}\right) dt$$

2.3 Кругова вставка

Загальний кут повороту між двома напрямами дороги позначимо θ. Тоді кут кругової дуги:

$$\varphi_{cir} = |\theta| - 2\,\varphi_{cl}$$

Умова існування фігури:

$$\varphi_{cir} \geq 0 \quad \Longleftrightarrow \quad |\theta| \geq 2\,\varphi_{cl} = \frac{S_0}{R}$$

Довжина кругової вставки:

$$L_{arc} = R\varphi_{cir}$$

Додатково в програмі використовується умова невиродженості дуги:

$$L_{arc} = L_{arc,\,\min}$$

Координати точок дуги:

$$P_{arc}(\phi) = C_{arc} + R\begin{pmatrix}\cos\phi \\ \sin\phi\end{pmatrix}, \quad \phi \in [a_{start},\, a_{end}]$$

2.4 Вихідна клотоїда

Вихідна клотоїда забезпечує плавний перехід від кривизни кола до нульової кривизни прямої. Її фазова функція має вигляд:

$$\phi_{out}(s) = \varphi_{cl} + \varphi_{cir} + \lambda s - \frac{A s^2}{2}$$

Координати вихідної клотоїди також обчислюються інтегрально::

$$x_{out}(s) = \int_0^s \cos\phi_{out}(t)\,dt, \qquad y_{out}(s) = \int_0^s \sin\phi_{out}(t)\,dt$$

2.5 Осьова прив'язка фігури

Фігура будується в локальній системі координат з горизонтальною вхідною дотичною і початком у (0,0). Для розміщення фігури в глобальній СК потрібно знайти відстань $W_{loc,x}$ від початку фігури до точки перетину вхідної і вихідної дотичних W_{loc}.

Ця точка знаходиться як перетин нормалі до бісектриси повного кута θ через середню точку дуги P_{mid} з горизонтальною віссю.

Середня точка дуги:

$$P_{mid} = C_{arc} + R \cdot \mathbf{t}(\phi_{mid}), \qquad \mathbf{t}\left(\phi_{mid}\right) = \left(\cos\phi, \sin\phi\right)$$

Нормаль до бісектриси кута (вектор, перпендикулярний до бісектриси):

$$\mathbf{n}_{bis} = sign \begin{pmatrix} -\sin\left(\frac{|\theta|}{2}\right) \\ \cos\left(\frac{|\theta|}{2}\right) \end{pmatrix}$$

Пряма через P_{mid} в напрямку $\mathbf{n}_{bis}$ перетинає вісь $y = 0$ в точці:

$$W_{loc,x} = P_{mid,x} - P_{mid,y} \cdot \frac{n_{bis,x}}{n_{bis,y}}$$

2.6 Розміщення фігури в глобальній системі координат

Маючи глобальну точку перетину двох дотичних W і вхідний напрям α_{in}, початок фігури:

$$\mathbf{\Phi}_{start} = W - W_{loc,x} \cdot \mathbf{t}(\alpha_{in})$$

Матриця повороту на кут α_{in}:

$$\mathbf{R}(\alpha_{in}) = \begin{pmatrix} \cos\alpha_{in} & -\sin\alpha_{in} \\ \sin\alpha_{in} & \cos\alpha_{in} \end{pmatrix}$$

Будь-яка точка фігури в глобальній СК:

$$P_{global} = \mathbf{\Phi}_{start} + \mathbf{R}(\alpha_{in}) \cdot P_{local}$$

Альтернативне розміщення від правого кінця (для другої фігури при схемі двох фігур):

$$\mathbf{\Phi}_{start} = T_{to} - \mathbf{R}(\alpha_{in}) \cdot P_{exit,end}, \qquad \mathbf{\Phi}_{end} = T_{to}$$

2.7 Знакова кутова функція і перетин прямих

Знаковий кут між двома напрямами, нормалізований у $(-\pi,\ \pi]$:

$$\theta = \text{signed_angle}(\alpha_{from}, \alpha_{to}) = \left[(\alpha_{to} - \alpha_{from} + \pi) \bmod 2\pi\right] - \pi$$

Точка перетину двох прямих: $P_1 + s_1\mathbf{t}_1$ і $P_2 + s_2\mathbf{t}_2$. Позначимо псевдоскалярний добуток $\mathbf{a} \times \mathbf{b} = a_x b_y - a_y b_x$. Тоді:

$$s_1 = \frac{(\Delta P \times \mathbf{t}_2)}{\mathbf{t}_1 \times \mathbf{t}_2}, \qquad s_2 = \frac{(\Delta P \times \mathbf{t}_1)}{\mathbf{t}_1 \times \mathbf{t}_2}, \qquad \Delta P = P_2 - P_1$$

$$W = P_1 + s_1\,\mathbf{t}_1$$

При $\left|\mathbf{t}_1 \times \mathbf{t}_2\right| < \varepsilon$ – прямі паралельні, перетину немає.

Умова коректного розміщення однієї фігури між T_{from} і T_{to}:

$$s_1 > 0 \quad (W \text{ попереду від } T_{from}) \quad \text{і} \quad s_2 < 0 \quad (W \text{ позаду від } T_{to})$$

2.8 Алгоритм двох фігур

Коли одна фігура не вміщується між T_{from} і T_{to}, будуються дві послідовні фігури $\Phi_2 : \left(\alpha_{from} \to \alpha_B\right)$ та $\Phi_3 : \left(\alpha_B \to \alpha_{to}\right)$ з невідомим проміжним напрямом α_B.

Кінець Φ_2 прив’язується до T_{to} ліворуч:

$$\mathbf{\Phi}_2^{end} = T_{from} + W_{loc,x}^{(2)} \cdot \mathbf{t}(\alpha_{from}) + \mathbf{R}(\alpha_{from}) \cdot P_{exit,end}^{(2)}$$

Початок Φ_3 прив’язується до T_{to} праворуч:

$$\mathbf{\Phi}_3^{start} = T_{to} - \mathbf{R}(\alpha_B) \cdot P_{exit,end}^{(3)}$$

Умова замикання – вектор «мосту» $\delta = \mathbf{\Phi}_3^{start} - \mathbf{\Phi}_2^{end}$ повинен бути спрямований вздовж $\mathbf{t}\left(\alpha_B\right)$, тобто поперечна компонента нульова:

$$f(\alpha_B) = \delta(\alpha_B) \times \mathbf{t}(\alpha_B) = 0, \qquad \delta(\alpha_B) \cdot \mathbf{t}(\alpha_B) \geq 0$$

2.9 Побудова сегмента дороги між двома містами

Для побудови переходу між двома сусідніми містами спочатку визначаються точки виходу з робочих кіл міст у напрямах дороги. Пряма в параметричному вигляді задається як:

$$P(t) = P_0 + t\mathbf{d}, \quad t \geq 0$$

Щоб знайти точку виходу прямої з кола з центром C і робочим радіусом $r^{contact}$, використовується умова:

$$\|P(t) - C\|^2 = r_{contact}^2$$

Після підстановки параметричного рівняння прямої отримується квадратне рівняння відносно параметра t. Серед коренів обирається найбільший невід’ємний, що дає точку виходу з кола.

Для забезпечення місця під побудову клотоїдної фігури використовується буферна відстань:

$$L_{buf} = \min\left(L_{buf}, 0.45 \cdot \|T_{i+1} - T_i\|\right)$$

Точки початку і кінця фігури між містами:

$$P_{exit}^{(i)} = P_{exit,base}^{(i)} + L_{buf} \cdot \mathbf{t}(\alpha_i)$$

$$P_{entry}^{(i+1)} = P_{entry,base}^{(i+1)} - L_{buf} \cdot \mathbf{t}(\alpha_{i+1})$$

Повна структура одного сегменту між містами:

$$T_i \xrightarrow{\text{пряма}} P_{exit}^{(i)} \xrightarrow{\text{фігура(и)}} P_{entry}^{(i+1)} \xrightarrow{\text{пряма}} T_{i+1}$$

2.10 Перевірка зіткнення траси з зоною міста

Перевірка, чи перетинає відрізок $[P_1, P_2]$ коло. Параметрична точка відрізка:

$$P(t) = P_1 + td, \quad \mathbf{d} = P_2 - P_1, \quad t \in [0,1]$$

Підстановка в $\|P(t) - C\|^2 = r^2$ дає квадратне рівняння:

$$a\,t^2 + b\,t + c = 0, \text{ де}$$

$$a = \|\mathbf{d}\|^2, \quad b = 2(\mathbf{f} \cdot \mathbf{d}), \quad c = \|\mathbf{f}\|^2 - r^2, \quad \mathbf{f} = P_1 - C$$

Зіткнення є якщо $D \geq 0$ і виконується хоча б одна з умов:

$t_1 \in [0,1]$ або $t_2 \in [0,1]$ або $(t_1 < 0$ і $t_2 > 1)$

Остання умова відповідає випадку, коли відрізок повністю лежить всередині кола. Для криволінійних ділянок використовується дискретний контроль: перевіряється, чи не потрапляє хоча б одна з обчислених точок в заборонену область. Такий самий підхід застосовується як для міст, так і для озер., але радіус збільшується на поле безпеки $margin = 5$ км, і повертається перша перешкода вздовж відрізка $M_i \to M_{i+1}$.

2.11 Обхід перешкод

Якщо відрізок перетинає перешкоду (озеро), між містами вставляється допоміжне «місто-озеро» з радіусом контакту:

$$r_{obs}^{contact} = r_{obs} + \Delta_{bypass}, \qquad \Delta_{bypass} = 2.0 \text{ км}$$

Допоміжний вузол задається центром озера, збільшеним радіусом та локальним напрямом обходу:

$$L = \left(C_{lake}, r_{obs}^{contact}, \alpha_{lake}\right),$$

де напрям береться за напрямом між сусідніми вузлами:

$$\alpha_{lake} = \arctan 2\left(y_{i+1} - y_i,\ x_{i+1} - x_i\right)$$

Якщо на одній ділянці траси є кілька озер, вони можуть вставлятися послідовно в порядку їх розташування вздовж ділянки. Після цього дорога перебудовується вже з урахуванням нових допоміжних вузлів.

У програмі також реалізовано захист від зациклення при побудові обходу перешкод: кожне озеро може бути вставлене лише один раз, а кількість допоміжних вузлів і час побудови обмежуються. Це дозволяє уникнути зависання програми у випадку складної або вкладеної конфігурації озер.

2.12 Оптимізація порядку обходу міст

Задача вибору оптимального порядку обходу n міст є різновидом задачі комівояжера. Використовується комбінований евристичний підхід.

Евклідова відстань між двома містами:

$$d(M_i, M_j) = \|M_i^{center} - M_j^{center}\|$$

Для певного порядку відвідування міст евклідова довжина маршруту обчислюється як:

$$L_{euclid}(\pi) = \sum_{k=0}^{n-2} d(M_{\pi_k}, M_{\pi_{k+1}})$$

У програмі початкові маршрути формуються методом найближчого сусіда: на кожному кроці обирається невідвідане місто з мінімальною відстанню до поточного.

2-opt покращення: для кожної пари індексів $1 \le i < j \le n-1$ перевіряється умова покращення при переворотові підмаршруту $[\pi_i \dots \pi_{j-1}]$:

$$d(\pi_{i-1}, \pi_i) + d(\pi_{j-1}, \pi_j) > d(\pi_{i-1}, \pi_{j-1}) + d(\pi_i, \pi_j)$$

Якщо виконується – підмаршрут перевертається. Після цього кілька найкращих маршрутів за евклідовою оцінкою перевіряються вже за реальною довжиною побудованої дороги, яка враховує клотоїди, кругові дуги, точки дотику та можливі перешкоди. Такий підхід дозволяє отримати якісний порядок обходу міст без повного перебору всіх перестановок.

3 ПРОГРАМНА РЕАЛІЗАЦІЯ

Програму реалізовано мовою Python. У поточній версії вся основна логіка побудови дороги зосереджена в одному файлі. Такий підхід спрощує структуру проєкту, полегшує перевірку правильності формул та робить програму зручнішою для аналізу в межах лабораторної роботи.

Програма складається з кількох логічних частин. Перша частина містить геометричні допоміжні функції: обчислення одиничних векторів, нормалей, знакового кута, перетину прямих, перевірки перетину відрізка з колом та службові операції для побудови фігури в локальній і глобальній системах координат. Друга частина відповідає за реалізацію математичної моделі клотоїдної фігури клотоїда – коло – клотоїда, зокрема за побудову вхідної клотоїди, кругової вставки та вихідної клотоїди. Третя частина виконує побудову сегмента між двома містами, у тому числі обробляє випадок, коли однієї фігури недостатньо і необхідно використати дві фігури з проміжною прямою. Четверта частина реалізує обхід перешкод, зокрема озер, шляхом вставки допоміжних вузлів маршруту. П’ята частина відповідає за оптимізацію порядку обходу міст. Окремо реалізовано графічний інтерфейс користувача та візуалізацію результатів.

Основними функціями програми є build_figure(…), build_segment(…), build_road(…), inject_obstacles(…), optimize_route(…) та _plot_onto_ax(…):

- Функція build_figure(…) будує локальну геометрію фігури клотоїда – коло – клотоїда.
- Функція build_segment(…) формує криволінійний перехід між двома напрямами.
- Функція build_road(…) об’єднує окремі сегменти в повну трасу через послідовність міст.
- Функція inject_obstacles(…) вставляє допоміжні вузли для обходу озер.
- Функція optimize_route(…) підбирає порядок обходу міст.

- Функція _plot_onto_ax(…) відповідає за графічне відображення міст, озер, точок дотику та самої дороги.

Графічний інтерфейс дозволяє користувачу вводити та редагувати координати міст, їхні радіуси, напрямки дороги та сторону обходу. Також можна додавати та редагувати озера, змінювати параметри, обирати стартове місто та запускати повторну побудову дороги. Результат відображається на графіку у вигляді міст, перешкод, точок дотику, прямих ділянок, вхідних і вихідних клотоїд та кругових дуг. Приклад інтерфейсу, зображений на рис. 2.

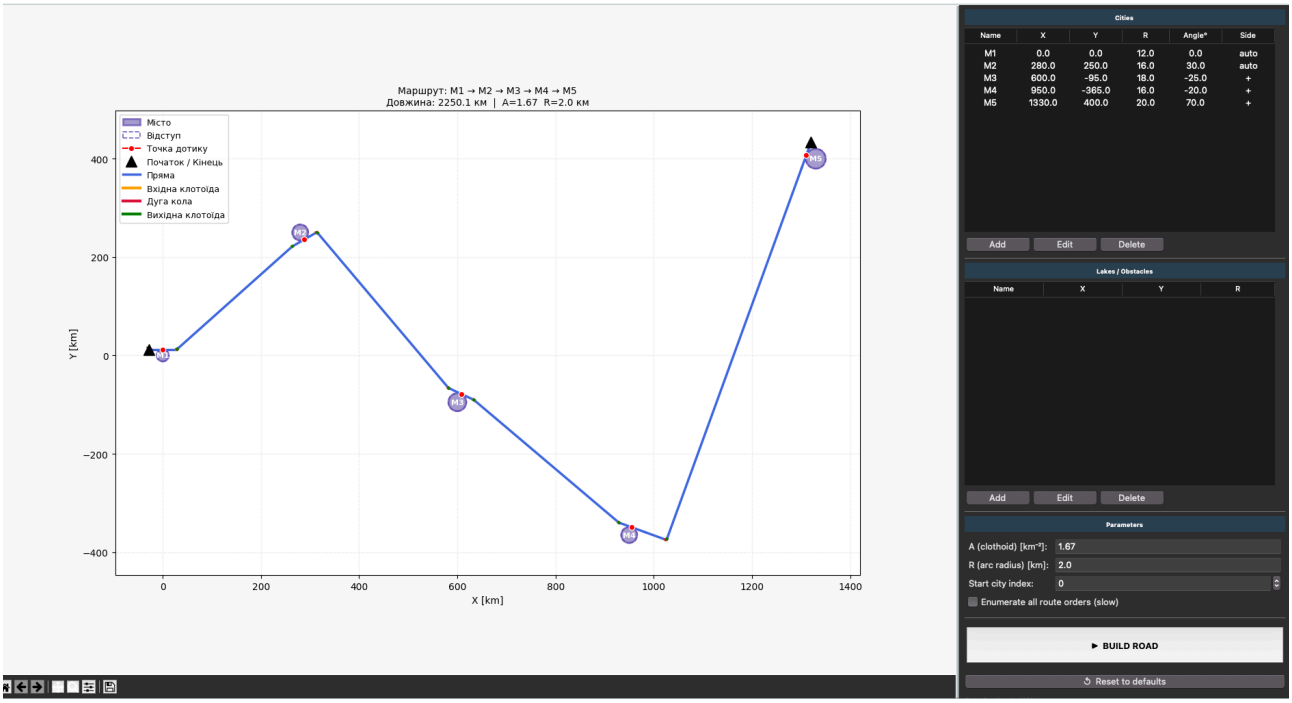


Рис. 2 – Інтерфейс програми

4 ПРИКЛАДИ РОБОТИ ПРОГРАМИ

Для перевірки працездатності програми було розглянуто кілька типових сценаріїв побудови дороги. Перший сценарій відповідає побудові маршруту між містами без перешкод. У цьому випадку програма формує повну трасу через усі задані міста, визначає точки дотику до робочих кіл міст, будує прямі ділянки та криволінійні переходи між ними. Результат показує, що траса не заходить у заборонені зони міст і має плавну геометрію.

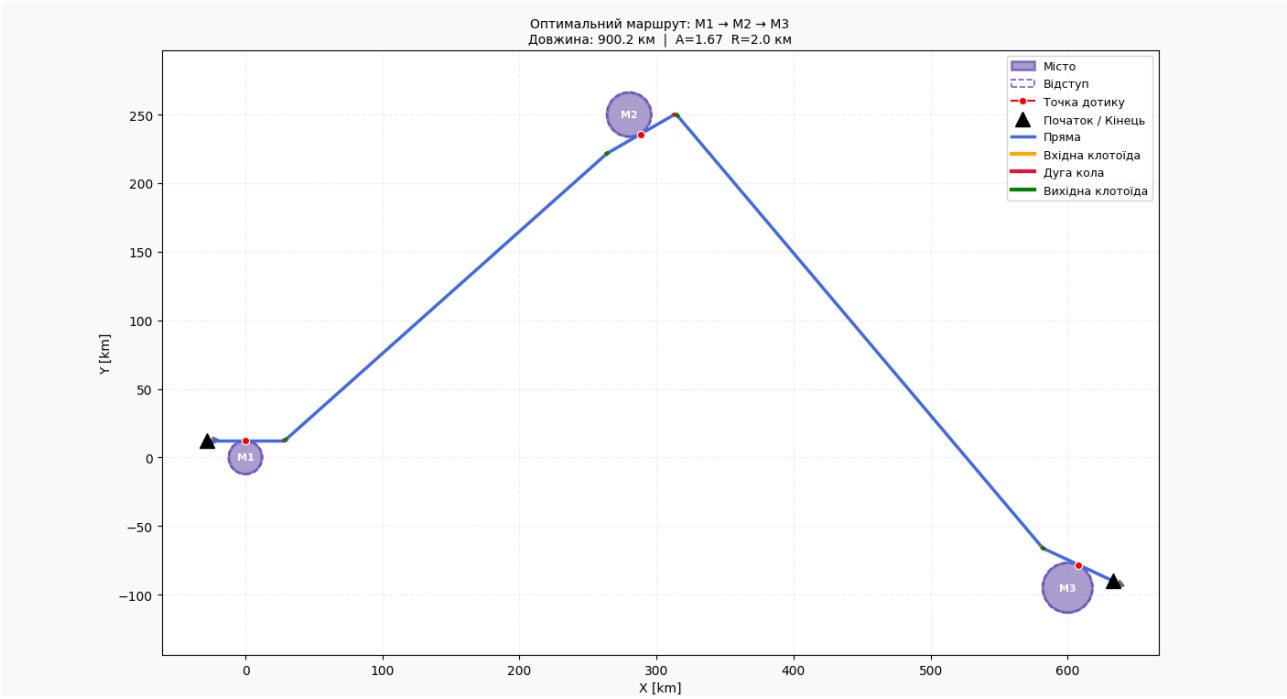


Рис. 3 – Побудова дороги між містами без перешкод

Другий сценарій демонструє врахування однієї перешкоди у вигляді озера. Якщо озеро розташоване на шляху побудованої дороги, програма додає допоміжний вузол обходу та перебудовує маршрут так, щоб дорога проходила поза межами збільшеного радіуса озера. У результаті змінюється конфігурація траси та збільшується її загальна довжина, однак вимога неперетину перешкоди зберігається.

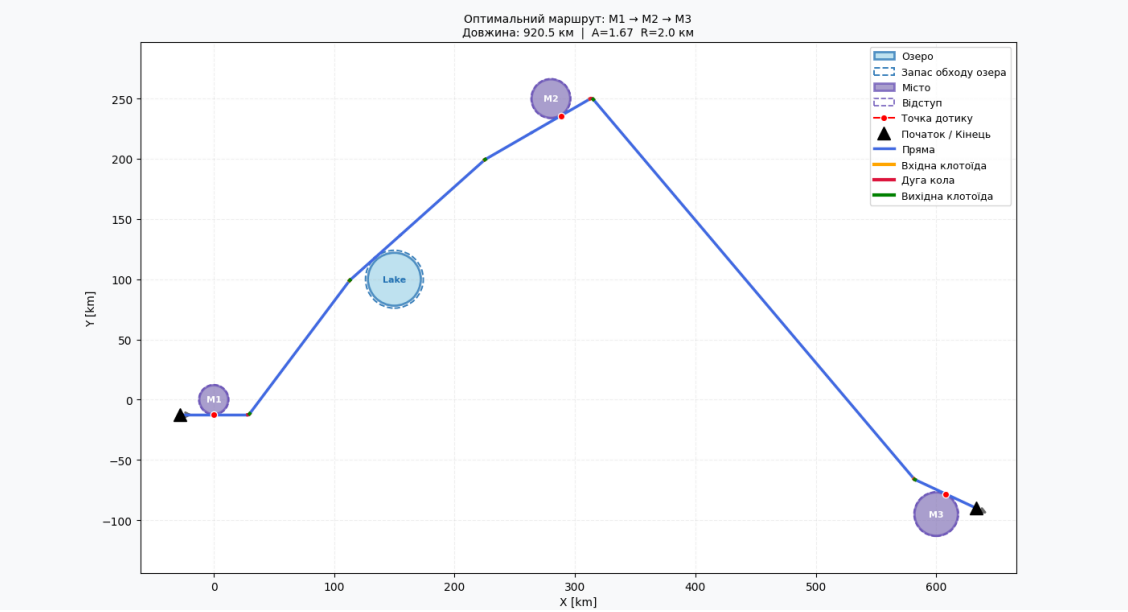


Рис. 4 – Побудова дороги з обходом одного озера

Третій сценарій пов’язаний із випадком кількох перешкод. Якщо на одній ділянці траси розташовано кілька озер, програма може вставляти кілька допоміжних вузлів послідовно. Це дозволяє побудувати складніший маршрут без перетину заборонених зон. Для запобігання зацикленню в алгоритмі передбачено захист від надмірної кількості вставок і перевищення допустимого часу обчислення.

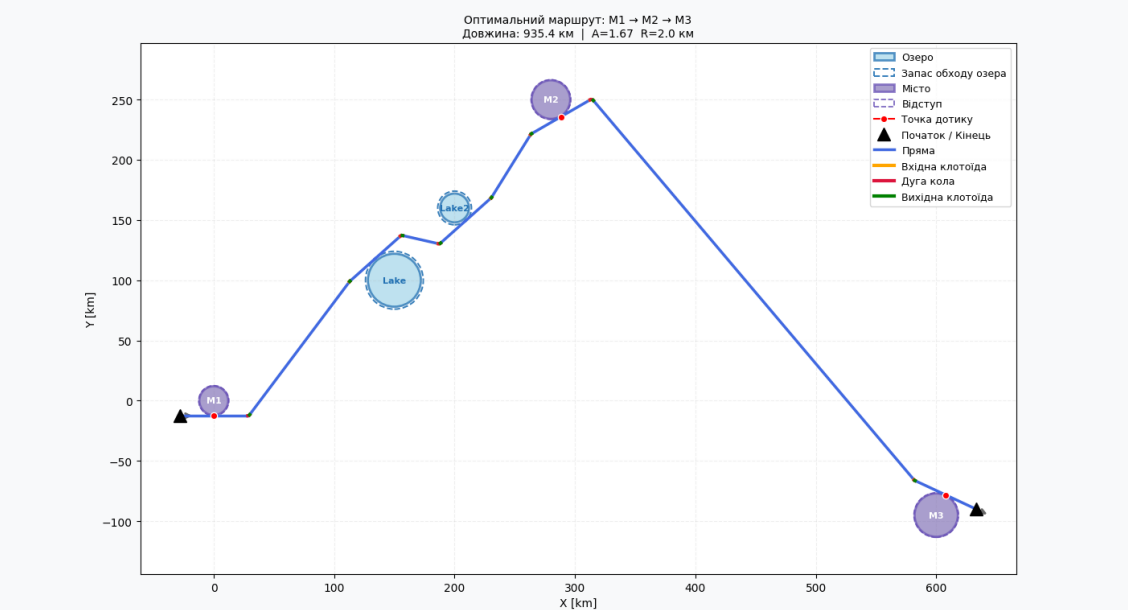


Рис. 5 – Побудова дороги з послідовним обходом кількох озер

Четвертий сценарій демонструє вплив сторони обходу міста. Якщо користувач змінює параметр road_side, змінюється точка дотику траси до міста, а отже, і форма всієї дороги на прилеглих ділянках. Це дозволяє дослідити, як локальна зміна геометрії в одному вузлі впливає на глобальний вигляд маршруту.

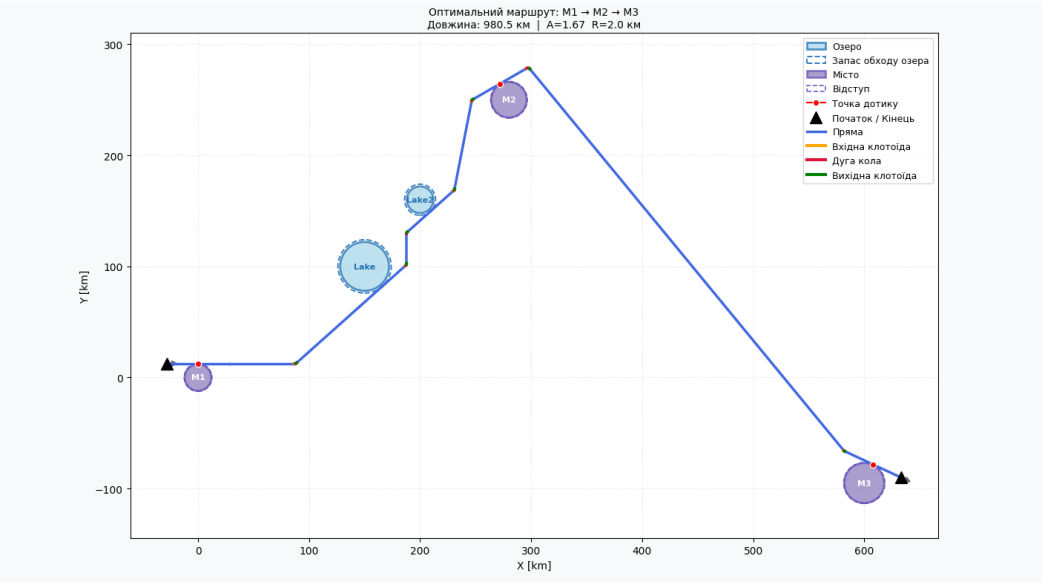


Рис. 6 – Вплив зміни сторони обходу міста на форму траси

П’ятий сценарій демонструє вплив параметрів R і A. При зміні цих параметрів змінюється довжина клотоїд, форма кругових вставок і загальна конфігурація траси. Для деяких поєднань параметрів побудова може стати неможливою, якщо класична фігура не задовольняє умову існування.

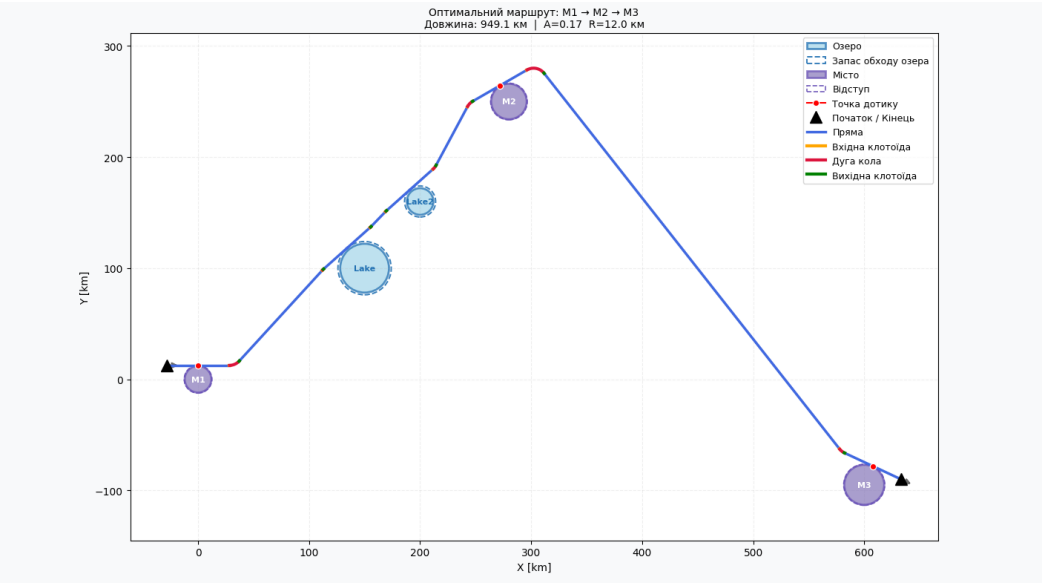


Рис. 7 – Вплив параметрів R і A на побудову дороги

5 АНАЛІЗ РЕЗУЛЬТАТІВ

Отримані результати показують, що реалізована програма коректно відтворює класичну схему побудови дороги на основі фігури клотоїда – коло – клотоїда. Використання інтегралів Френеля для обчислення координат клотоїди забезпечує точну побудову криволінійних ділянок і відповідає теоретичній постановці задачі.

Аналіз показав, що при помірних значеннях кута повороту між сусідніми напрямами дорога може бути побудована за допомогою однієї класичної фігури. Якщо ж геометрія переходу є складнішою, програма використовує схему з двох фігур і проміжної прямої. У такому випадку проміжний напрям визначається чисельно з умови сумісності частин траси.

Окрему увагу було приділено випадкам різких поворотів поблизу міста. Практика показала, що в таких конфігураціях короткої прямої вставки перед початком повороту може бути недостатньо. Тому в програмі реалізовано адаптивне подовження прямої вставки, що дозволяє уникнути входження траси в заборонену зону міста. Це покращує геометричну коректність побудови без зміни основної теоретичної моделі клотоїдної фігури.

Наявність перешкод у вигляді озер значно ускладнює задачу, оскільки маршрут має узгоджуватись не лише з містами, а й з додатковими забороненими зонами. У програмі для цього використовується вставка допоміжних вузлів маршруту. При кількох озерах на одній ділянці вони можуть додаватись послідовно. При цьому було встановлено, що для складних конфігурацій необхідний механізм захисту від зациклення. Такий механізм у програмі реалізовано шляхом обмеження повторних вставок, кількості допоміжних вузлів та часу обчислення.

Ще одним важливим спостереженням є вплив параметрів R і A на можливість побудови дороги. При деяких поєднаннях цих параметрів класична фігура не існує, оскільки кругова вставка стає виродженою або вся побудова не вміщується в геометрію задачі. У таких випадках програма коректно повідомляє про неможливість побудови.

Оптимізація порядку обходу міст дозволяє зменшувати загальну довжину маршруту. Однак у поточній реалізації вона не є точним повним перебором усіх перестановок, а використовує комбінований евристичний підхід: формування кандидатних маршрутів на базі найближчого сусіда, локальне покращення методом 2-opt та перевірку кількох найкращих варіантів за реальною довжиною вже побудованої дороги. Це забезпечує прийнятний компроміс між швидкістю та якістю результату.

6 ВИСНОВКИ

У ході лабораторної роботи було реалізовано програму побудови автомобільної дороги між містами на основі класичної схеми клотоїда – коло – клотоїда. У математичній моделі використано лінійний закон зміни кривизни клотоїди, а координати кривої обчислюються через інтеграли Френеля.

У програмі реалізовано побудову траси через кілька міст, визначення точок дотику до робочих кіл міст, побудову однієї або двох клотоїдних фігур між сусідніми напрямами, а також врахування перешкод у вигляді озер. Додатково реалізовано зміну сторони обходу міста, підбір порядку обходу міст та захист від зациклення при складній конфігурації перешкод.

У результаті перевірки на різних прикладах встановлено, що програма забезпечує побудову плавної траси, яка не перетинає заборонені зони міст і перешкод за умови існування геометрично допустимого розв’язку. Для різких поворотів було реалізовано адаптивне подовження прямої вставки перед фігурою, що дозволило уникнути проходження дороги через місто.

Таким чином, поставлену мету роботи досягнуто: побудовано програмний засіб для геометричного моделювання дороги між містами, досліджено застосування клотоїдних переходів і перевірено працездатність алгоритму на низці тестових конфігурацій.

7 СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Диференційна Геометрія і Сплайни – криві та поверхні [Електронний ресурс]: навч. посіб. для студ. спеціальності 113 «Прикладна математика»/ КПІ ім. Ігоря Сікорського; уклад.: І.В.Ориняк. – Електронні текстові дані. – Київ : КПІ ім. Ігоря Сікорського, 2025. – 252 с.
2. Bertolazzi E., Frego M. Fast and accurate G1 fitting of clothoid curves. arXiv preprint. 2013. arXiv:1305.6644. URL: https://arxiv.org/pdf/1305.6644 (дата звернення: 17.04.2026).
3. Lichtenberg G., Halvorsen S. Railway transition curves: a review of the state-of-the-art and future research. Infrastructures. 2020. Vol. 5, No. 5. Art. 43. URL: https://www.mdpi.com/2412-3811/5/5/43 (дата звернення: 12.04.2026).
4. Walz G. Spline-funktionen als lösungen von randwertaufgaben. Mathematics of Computation. 1992. Vol. 59, No. 199. P. 125–130. URL: https://www.ams.org/journals/mcom/1992-59-199/S0025-5718-1992-1134736-8/S0025-5718-1992-1134736-8.pdf (дата звернення: 29.04.2026).

ДОДАТОК А

У цьому додатку наведено основні фрагменти програмної реалізації, які відповідають ключовим етапам побудови автомобільної дороги між містами.

А.1 Побудова клотоїдної фігури

```
def build_figure(alpha_in, alpha_out, R_c, A_c, n=600):
    core = _figure_core_cached(alpha_in, alpha_out, R_c, A_c)
    theta = core["theta"]
    sign = core["sign"]
    S0 = core["S0"]
    phi_cl = core["phi_cl"]
    phi_cir = core["phi_cir"]
    lam = 1.0 / R_c
    n = _adaptive_plot_samples(abs(theta), base_count=n)
    s = np.linspace(0.0, S0, n)

    Xin, Yin = _clothoid_xy_exact(A_c, s, sign)
    pts_in = np.column_stack([Xin, Yin])

    P_in_end = core["P_in_end"].copy()
    C_arc = core["C_arc"].copy()
    P_arc_end = core["P_arc_end"].copy()
    angle_arc_end = core["angle_arc_end"]
    v0 = P_in_end - C_arc
    a_start = np.arctan2(v0[1], v0[0])
    a_end = np.arctan2(P_arc_end[1] - C_arc[1], P_arc_end[0] - C_arc[0])
    angs = np.linspace(a_start, a_end, n)
    pts_arc = C_arc + R_c * np.column_stack([np.cos(angs), np.sin(angs)])

    Xex, Yex = _phase_integral(-sign * A_c / 2.0, sign * lam, angle_arc_end, s)
    pts_exit = P_arc_end + np.column_stack([Xex, Yex])

    return {
        **core,
        "pts_in": pts_in,
        "pts_arc": pts_arc,
        "pts_exit": pts_exit,
    }
```

А.2 Побудова сегмента дороги між двома містами

```
def build_segment(T_from, alpha_from, T_to, alpha_to, R_c, A_c):
    if np.linalg.norm(T_to - T_from) < 1e-6:
        return []

    alpha_direct = np.arctan2(*(T_to - T_from)[::-1])
    if abs(signed_angle(alpha_from, alpha_to)) < 1e-6 and abs(signed_angle(alpha_from,
alpha_direct)) < 1e-6:
        return [{"type": "straight", "p_start": T_from, "p_end": T_to}]

    use_one, W = can_use_one_figure(T_from, alpha_from, T_to, alpha_to, R_c, A_c)
    if use_one:
        fig = place_figure(build_figure(alpha_from, alpha_to, R_c, A_c), W, unit(alpha_from),
alpha_from)
        return [
            {"type": "straight", "p_start": T_from, "p_end": fig["Phi_start"]},
            {"type": "figure", **fig, "alpha_in": alpha_from, "alpha_out": alpha_to},
            {"type": "straight", "p_start": fig["Phi_end"], "p_end": T_to},
        ]

    alpha_B, bridge_len = _solve_alpha_B(T_from, alpha_from, T_to, alpha_to, R_c, A_c)
    fig2 = place_figure(
        build_figure(alpha_from, alpha_B, R_c, A_c),
        T_from + _figure_core_cached(alpha_from, alpha_B, R_c, A_c)["W_loc_x"] * unit(alpha_from),
        unit(alpha_from),
        alpha_from,
```

```
    )
    fig3 = place_figure_right(build_figure(alpha_B, alpha_to, R_c, A_c), T_to, alpha_B)
    bridge_end = fig2["Phi_end"] + bridge_len * unit(alpha_B)
    return [
        {"type": "figure", **fig2, "alpha_in": alpha_from, "alpha_out": alpha_B},
        {"type": "straight", "p_start": fig2["Phi_end"], "p_end": bridge_end, "is_B": True},
        {"type": "figure", **fig3, "alpha_in": alpha_B, "alpha_out": alpha_to},
    ]
```

A.3 Пошук проміжного напряму для двох фігур

```
def _solve_alpha_B(T_from, alpha_from, T_to, alpha_to, R_c, A_c):
    alpha_direct = np.arctan2(*(T_to - T_from)[::-1])

    def residual(angle):
        cross, _ = _two_figure_trial(T_from, alpha_from, T_to, alpha_to, R_c, A_c, angle)
        return cross

    def scan_grid(span, count):
        grid = alpha_direct + np.linspace(-span, span, count)
        samples = []
        for angle in grid:
            try:
                samples.append((angle, *_two_figure_trial(T_from, alpha_from, T_to, alpha_to, R_c,
A_c, angle)))
            except ValueError:
                samples.append(None)

        roots = []
        for left, right in zip(samples, samples[1:]):
            if left is None or right is None:
                continue
            a0, f0, _ = left
            a1, f1, _ = right
            if abs(f0) < 1e-8:
                roots.append(a0)
                continue
            if f0 * f1 > 0:
                continue
            try:
                roots.append(brentq(residual, a0, a1, xtol=1e-10, rtol=1e-10, maxiter=100))
            except ValueError:
                pass
        return roots

    candidates = []
    for span, count in (
        (np.pi / 3.0, 241),
        (2.0 * np.pi / 3.0, 361),
        (np.pi - 1e-4, 721),
    ):
        candidates = scan_grid(span, count)
        if candidates:
            break

    best = None
    for alpha_B in candidates:
        try:
            cross, dot = _two_figure_trial(T_from, alpha_from, T_to, alpha_to, R_c, A_c, alpha_B)
        except ValueError:
            continue
        if abs(cross) > 1e-7 or dot < -1e-7:
            continue
        item = (
            np.linalg.norm(T_to - T_from) + max(dot, 0.0),
            abs(signed_angle(alpha_B, alpha_direct)),
            alpha_B,
            dot,
        )
        if best is None or item < best:
            best = item

    if best is None:
        raise ValueError(
            "The classical clothoid-straight-clothoid transition does not exist "
            f"for the chosen geometry and fixed parameters R={R_c:.4g}, A={A_c:.4g}."
        )
```

```
    return best[2], max(best[3], 0.0)
```

A.4 Перевірка перетину траси з містом або перешкодою

```
def _segment_intersects_circle(p_start, p_end, center, radius, tol=0.1):
    d = p_end - p_start
    f = p_start - center
    a = np.dot(d, d)
    if a < 1e-12:
        return np.linalg.norm(f) < radius - tol
    b = 2.0 * np.dot(f, d)
    c = np.dot(f, f) - (radius - tol) ** 2
    disc = b * b - 4.0 * a * c
    if disc < 0:
        return False
    sq = np.sqrt(disc)
    t1 = (-b - sq) / (2.0 * a)
    t2 = (-b + sq) / (2.0 * a)
    return (0.0 <= t1 <= 1.0) or (0.0 <= t2 <= 1.0) or (t1 < 0.0 and t2 > 1.0)


def check_collision(segments, city_center, radius):
    for seg in segments:
        if seg["type"] == "straight":
            if _segment_intersects_circle(seg["p_start"], seg["p_end"], city_center, radius):
                return True
        else:
            for key in ("pts_in_g", "pts_arc_g", "pts_exit_g"):
                pts = seg[key]
                if np.any(np.linalg.norm(pts - city_center, axis=1) < radius - 0.1):
                    return True
    return False


def segment_collides_with_circle(seg, center, radius):
    if seg["type"] == "straight":
        return _segment_intersects_circle(seg["p_start"], seg["p_end"], center, radius)
    for key in ("pts_in_g", "pts_arc_g", "pts_exit_g"):
        pts = seg[key]
        if np.any(np.linalg.norm(pts - center, axis=1) < radius - 0.1):
            return True
    return False
```

A.5 Обхід озер

```
def inject_obstacles(ordered_cities, obstacles, R_c=None, A_c=None):
    if R_c is None:
        R_c = DEFAULT_R
    if A_c is None:
        A_c = DEFAULT_A

    final_path = [copy.deepcopy(city) for city in ordered_cities]
    inserted_keys = {_obstacle_key(c) for c in final_path if c.get("is_lake")}
    max_insertions = max(8, len(obstacles) * MAX_OBSTACLE_INSERTIONS_FACTOR + len(ordered_cities))
    insertions = len(inserted_keys)
    started = time.perf_counter()

    while True:
        if insertions > max_insertions:
            raise ValueError(
                "Обхід перешкод зупинено: надто багато допоміжних вузлів для озер. "
                "Ймовірно, конфігурація перешкод викликає зациклення або є занадто складною."
            )
        if time.perf_counter() - started > MAX_OBSTACLE_BUILD_SECONDS:
            raise ValueError(
                "Обхід перешкод зупинено через перевищення часу. "
                "Спробуй прибрати вкладені озера або спростити конфігурацію."
            )

        inserted = False

        for i in range(len(final_path) - 1):
            current_city = final_path[i]
```

```
            next_city = final_path[i + 1]
            blocked_list = blocked_with_margin_sorted(current_city["center"], next_city["center"],
obstacles)
            blocked_list = [
                obs
                for _, obs in blocked_list
                if not (
                    _obstacle_key(obs) in inserted_keys
                    or
                    (current_city.get("is_lake") and np.allclose(current_city["center"],
obs["center"]))
                    or (next_city.get("is_lake") and np.allclose(next_city["center"],
obs["center"]))
                )
            ]
            if not blocked_list:
                continue

            diff = next_city["center"] - current_city["center"]
            opt_a = np.degrees(np.arctan2(diff[1], diff[0]))
            for blocked_by in reversed(blocked_list):
                key = _obstacle_key(blocked_by)
                final_path.insert(
                    i + 1,
                    {
                        "name": blocked_by.get("name", "Lake"),
                        "center": blocked_by["center"],
                        "radius": blocked_by["radius"] + LAKE_BYPASS_MARGIN,
                        "road_angle": opt_a,
                        "is_lake": True,
                    },
                )
                inserted_keys.add(key)
                insertions += 1
            inserted = True
            break

        if inserted:
            continue
        try:
            segments, _ = build_road(copy.deepcopy(final_path), R_c, A_c)
        except Exception:
            break

        collision_found = None
        for obs in obstacles:
            if _obstacle_key(obs) in inserted_keys:
                continue
            for seg in segments:
                if segment_collides_with_circle(seg, obs["center"], obs["radius"]):
                    pair_idx = seg.get("pair_idx")
                    if pair_idx is None or pair_idx >= len(final_path) - 1:
                        continue
                    collision_found = (pair_idx, obs)
                    break
            if collision_found is not None:
                break

        if collision_found is None:
            break

        pair_idx, obs = collision_found
        current_city = final_path[pair_idx]
        next_city = final_path[pair_idx + 1]
        diff = next_city["center"] - current_city["center"]
        opt_a = np.degrees(np.arctan2(diff[1], diff[0]))
        key = _obstacle_key(obs)
        final_path.insert(
            pair_idx + 1,
            {
                "name": obs.get("name", "Lake"),
                "center": obs["center"],
                "radius": obs["radius"] + LAKE_BYPASS_MARGIN,
                "road_angle": opt_a,
                "is_lake": True,
            },
        )
        inserted_keys.add(key)
        insertions += 1
    return final_path
```

А.6 Оптимізація порядку обходу міст

```
def nearest_neighbor(cities, start_idx=0):
    n = len(cities)
    order = [start_idx]
    visited = {start_idx}
    current = start_idx
    while len(order) < n:
        best_d, best_j = float("inf"), -1
        for j in range(n):
            if j in visited:
                continue
            d = euclid_dist(cities[current], cities[j])
            if d < best_d:
                best_d, best_j = d, j
        if best_j == -1:
            break
        order.append(best_j)
        visited.add(best_j)
        current = best_j
    return order


def two_opt(order, cities):
    best = order[:]
    improved = True
    while improved:
        improved = False
        for i in range(1, len(best) - 2):
            for j in range(i + 1, len(best)):
                if j - i == 1:
                    continue
                candidate = best[:]
                candidate[i:j] = reversed(candidate[i:j])
                if total_euclid(candidate, cities) + 1e-9 < total_euclid(best, cities):
                    best = candidate
                    improved = True
        order = best
    return best

def optimize_route(cities, start_idx=0, R_c=DEFAULT_R, A_c=DEFAULT_A, obstacles=None,
exact_top_k=2):
    n = len(cities)
    free = [i for i in range(n) if i != start_idx]
    candidates = []

    nn_order = nearest_neighbor(cities, start_idx)
    candidates.append(two_opt(nn_order, cities))

    for second in free:
        rest = [x for x in free if x != second]
        visited = {start_idx, second}
        order = [start_idx, second]
        current = second
        for _ in range(len(rest)):
            best_d, best_j = float("inf"), -1
            for j in rest:
                if j not in visited:
                    d = euclid_dist(cities[current], cities[j])
                    if d < best_d:
                        best_d, best_j = d, j
            if best_j == -1:
                break
            order.append(best_j)
            visited.add(best_j)
            current = best_j
        candidate = two_opt(order, cities)
        if candidate not in candidates:
            candidates.append(candidate)

    for last in free:
        rest = [x for x in free if x != last]
        nn_sub = nearest_neighbor([cities[i] for i in [start_idx] + rest], 0)
        order = [([start_idx] + rest)[i] for i in nn_sub] + [last]
        candidate = two_opt(order, cities)
        if candidate not in candidates:
            candidates.append(candidate)
```

```
    candidates.sort(key=lambda o: total_euclid(o, cities))
    max_candidates = 12 if n <= 6 else 8
    max_candidates = max(max_candidates, exact_top_k * 4)
    top_candidates = candidates[: min(len(candidates), max_candidates)]

    best_order = None
    best_length = float("inf")
    for order in top_candidates:
        ordered = [copy.deepcopy(cities[i]) for i in order]
        if obstacles:
            ordered = inject_obstacles(ordered, obstacles, R_c=R_c, A_c=A_c)
        try:
            segs, _ = build_road(copy.deepcopy(ordered), R_c=R_c, A_c=A_c)
            length = road_length(segs)
            if length < best_length:
                best_length = length
                best_order = order
        except Exception:
            pass
    return best_order, best_length
```

A.7 Побудова повної дороги

```
def build_road(cities, R_c, A_c, L_buffer=DEFAULT_BUFFER_KM):
    cities = [copy.copy(c) for c in cities]
    T_points = [_resolve_tangent_point(cities, i) for i in range(len(cities))]
    all_segments = []

    for _ in range(5):
        changed = False
        for i, city in enumerate(cities):
            if city.get("_bypass_manual"):
                continue
            prev_city = cities[i - 1] if i > 0 else None
            next_city = cities[i + 1] if i < len(cities) - 1 else None
            prev_T = T_points[i - 1] if i > 0 else None
            next_T = T_points[i + 1] if i < len(cities) - 1 else None
            new_angle, new_T = _choose_bypass(
                city, prev_city, prev_T, next_city, next_T, R_c, A_c, L_buffer
            )
            if np.linalg.norm(new_T - T_points[i]) > 1e-6:
                changed = True
            city["bypass_angle"] = new_angle
            T_points[i] = new_T
        if not changed:
            break

    for i in range(len(cities) - 1):
        c_curr = cities[i]
        c_next = cities[i + 1]
        alpha_curr = np.radians(c_curr["road_angle"])
        alpha_next = np.radians(c_next["road_angle"])
        T_curr = T_points[i]
        T_next = T_points[i + 1]

        if i == 0:
            all_segments.append(
                {
                    "type": "straight",
                    "p_start": T_curr - L_buffer * unit(alpha_curr),
                    "p_end": T_curr,
                    "pair_idx": i,
                }
            )

        try:
            pair_segments = _build_pair_safe(c_curr, T_curr, c_next, T_next, R_c, A_c, L_buffer)
        except ValueError as exc:
            raise ValueError(
                "Classical transition is impossible in theoretical mode.\n"
                f"Segment: {_city_label(c_curr, i)} -> {_city_label(c_next, i + 1)}\n"
                f"Parameters: R={R_c:.4g}, A={A_c:.4g}, "
                f"turn angle={abs(np.degrees(signed_angle(alpha_curr, alpha_next))):.2f} deg\n"
                "This mode keeps R and A fixed and does not use heuristic re-fitting.\n"
                "Try one of these:\n"
                "- increase A;\n"
                "- decrease R;\n"
```

```
                "- change road side or bypass angle for one of the cities;\n"
                "- remove obstacles for the theoretical test.\n"
                f"Details: {exc}"
            ) from exc
        for seg in pair_segments:
            seg["pair_idx"] = i
        all_segments.extend(pair_segments)

        if i == len(cities) - 2:
            all_segments.append(
                {
                    "type": "straight",
                    "p_start": T_next,
                    "p_end": T_next + L_buffer * unit(alpha_next),
                    "pair_idx": i,
                }
            )

    return all_segments, T_points
```